

EfficientPBD: View-Frustum Culling for Real-Time Deformable Simulation in XR

Taeheon Kim^{1*} Hamin Lee^{1†} Nak-jun Sung^{2‡} Min Hong^{3§}

¹Department of Software Convergence, Soonchunhyang University

²Research Institute, National Cancer Center

³Department of Computer Software Engineering, Soonchunhyang University

ABSTRACT

Recent advances in mixed reality systems enable the reconstruction of real-world environments as dense scene meshes, allowing physical interaction between virtual objects and the surrounding space. However, direct collision handling between deformable objects and such meshes remains computationally expensive and can lead to unstable behavior in real-time XR applications. We propose EfficientPBD, a collision filtering framework that integrates view-frustum culling with budget-based triangle selection for real-time deformable simulation. By restricting collision processing to a selected subset of surface triangles in a GPU-based Position-Based Dynamics loop, EfficientPBD achieves up to a 74.2% improvement in frame rates for complex models.

Index Terms: Extended Reality, Deformable Simulation, Scene Mesh Collision, GPU PBD, View-Frustum Culling

1 INTRODUCTION

Modern XR headsets provide real-time spatial mapping, enabling virtual objects to collide with scene meshes [1]. However, these meshes are often dense and noisy, making stable interaction with deformable objects computationally challenging. Position-Based Dynamics (PBD) is widely used for its stability and has been applied to surface-based XR simulations [2]. Yet, volumetric objects require further optimization due to the high cost of full-mesh collision detection. Real-time collision handling in XR remains challenging due to noisy scene meshes. Early approaches such as KinectFusion focused on rigid interactions with reconstructed environments [3], while recent soft-body methods often rely on simplified collision proxies to maintain performance [4]. PBD is widely adopted for its numerical stability, and GPU-based formulations enable real-time simulation [5]; however, volumetric deformable objects remain computationally expensive in XR settings. Spatial acceleration techniques such as view-frustum culling have been explored mainly for rendering or rigid-body physics [6], but are rarely used as primary filters for deformable collision.

To address this challenge, we propose *EfficientPBD*, a real-time framework that combines view-frustum culling, strict triangle budgeting, and a GPU-based PBD solver to enable stable deformable interaction.

2 SYSTEM OVERVIEW

The architecture Figure 1 comprises three pipelined stages optimizing data transfer. (1) **Mesh Baking:** Raw XR meshes are transformed into world space and cached. (2) **Budgeted Culling:** The

CPU identifies visible surfaces via View-Frustum Culling. Relevant triangles are then selected based on proximity, capped by a budget K for consistent performance. (3) **GPU Simulation:** The PBD solver executes on the GPU, resolving static contacts as positional constraints to ensure penetration-free interaction.

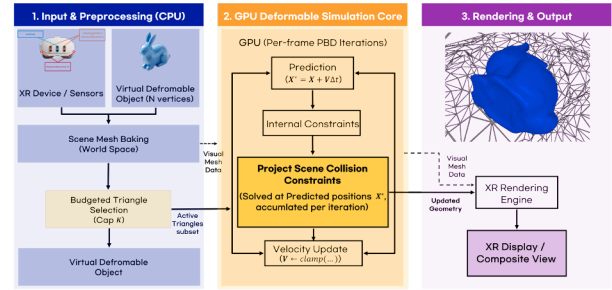


Figure 1: System architecture. The raw scene mesh is culled by the camera frustum, reduced to a budgeted collision proxy, and integrated into a GPU-PBD loop as positional constraints.

3 METHOD

3.1 View-Frustum Culling and Budgeted Selection

Direct collision against the entire Scene Mesh is unnecessary since users focus on interactions within their field of view. This study implements a two-pass selection strategy on the CPU before uploading data to the GPU:

1. **View-Frustum Culling:** First, all scene triangles lying outside the current camera frustum are culled. This drastically reduces the search space S_{total} to a potentially visible subset $S_{visible}$.
2. **Budgeted Allocation:** From $S_{visible}$, a final set of collision candidates is selected, limited by a hard cap K .

$$K_{active} = \min(|S_{visible}|, K) \quad (1)$$

3.2 Collision as Positional Constraints in PBD

Once the active triangles (K_{active}) are selected, physics simulation is performed on the GPU using Position-Based Dynamics (PBD) to achieve stable collision handling with noisy XR scene meshes.

Deformable objects are represented as tetrahedral meshes to support volumetric deformation and internal constraints, such as volume preservation, allowing collision responses to propagate throughout the object rather than being confined to surface geometry. Accordingly, vertex-triangle collision tests are applied to all tetrahedral nodes to preserve volumetric integrity during contact.

Continuous Collision Detection (CCD) is applied to node trajectories to prevent tunneling during fast motion. The simulation loop proceeds as follows:

*taecheon1213@sch.ac.kr

†haminprotoss@sch.ac.kr

‡njsung@ncc.re.kr

§mhong@sch.ac.kr

1. **Prediction:** Estimate the tentative position $\mathbf{x}^*$ using explicit Euler integration:

$$\mathbf{x}^* = \mathbf{x} + \Delta t \mathbf{v} + \Delta t^2 \mathbf{g} \quad (2)$$

2. **Constraint Projection:** Internal constraints (stretch, bend, volume) are solved first, followed by collision constraints. For every vertex intersecting a selected scene triangle:

$$\mathbf{x}^* \leftarrow \mathbf{x}^* + \Delta \mathbf{x}_{\text{collision}} \quad (3)$$

Here, $\Delta \mathbf{x}_{\text{collision}}$ projects the vertex to the nearest surface point of the triangle.

3. **Velocity Update:**

$$\mathbf{v} = (\mathbf{x}^* - \mathbf{x}) / \Delta t \quad (4)$$

4 IMPLEMENTATION AND RESULTS

4.1 Experimental Environment

The proposed system was implemented in Unity using GPU-based compute shaders and evaluated on a desktop PC connected to a Meta Quest 3 headset via Quest Link. All experiments were conducted using a reconstructed scene mesh with approximately $V_{\text{raw}} = 121,644$ vertices, representing a dense real-world environment. Due to the high computational cost of per-node collision checks in volumetric deformable simulation, the number of collision triangles was uniformly capped at $K = 10,000$ across all evaluations to ensure stable performance. The workstation was equipped with an AMD Ryzen Threadripper PRO 3955WX CPU, an NVIDIA GeForce RTX 4070 Ti GPU, and 128 GB of system memory.

4.2 Performance Analysis

The system was evaluated using three deformable models with increasing geometric complexity. All models were generated as tetrahedral meshes using TetGen to support volumetric deformation. Each experiment was repeated twice, and the reported values correspond to the average frame rate (FPS). Table 1 compares direct scene mesh collision with EfficientPBD under the same triangle budget ($K = 10,000$). All baseline and proposed methods were evaluated under identical and up-to-date hardware and software configurations to ensure a fair comparison. Due to the display refresh limit of the Meta Quest 3, the frame rate was capped at 120 FPS. EfficientPBD consistently achieves higher and more stable performance than direct collision, particularly as model complexity increases.

Table 1: Performance comparison in terms of frame rate (FPS) with and without view-frustum culling ($K = 10,000$).

Model	Nodes	Tets	NoCull.	EffPBD	Imp.(%)
Sphere	162	408	116	120	3.4
Torus	2,304	7,640	98	120	22.4
Bunny	2,990	10,724	66	115	74.2

4.3 Qualitative Results

Figure 2 illustrates the qualitative stability of the proposed approach during real-world interaction. Thanks to the constraint-based formulation of PBD, deformable objects settle stably on the reconstructed table surface without noticeable jitter, even in the presence of noisy scene mesh geometry. This indicates that the proposed method provides robust and visually plausible contact behavior for volumetric deformable objects in challenging XR environments.

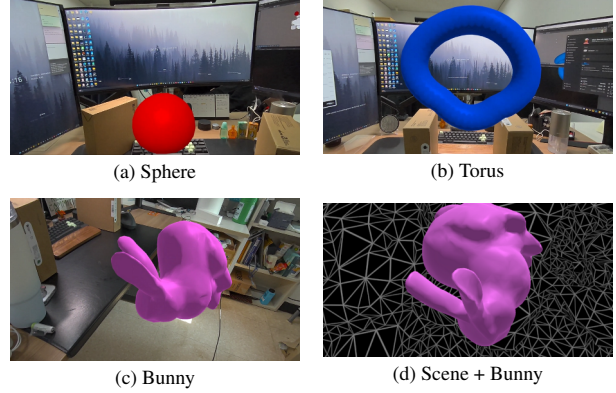


Figure 2: Qualitative results. (a–c) demonstrate stable tabletop contact for different volumetric shapes. (d) shows the Stanford Bunny interacting with a reconstructed real-world scene mesh.

5 CONCLUSION AND FUTURE WORK

This paper presented EfficientPBD, a collision filtering framework for real-time deformable interaction with dense scene meshes in XR. Experimental results show that combining view-frustum culling with a fixed triangle budget of 10K and a constraint-based GPU solver improves frame-rate stability across deformable objects of varying geometric complexity. The proposed approach indicates potential applicability to XR scenarios requiring interactive and physically plausible contact with complex surfaces, including medical and surgical simulation. Future work will focus on optimizing vertex compaction and memory management for standalone XR devices, as well as developing adaptive triangle budgeting strategies to balance simulation quality and performance.

ACKNOWLEDGMENTS

This research was supported by the Basic Science Research Program through the National Research Foundation of Korea (NRF), funded by the Ministry of Education (NRF-2022R111A3069371), funded by BK21 FOUR (Fostering Outstanding Universities for Research) (No.: 5199990914048), and supported by the Soonchunhyang University Research Fund.

REFERENCES

- [1] Xingdong Sheng, Shijie Mao, Yichao Yan, and Xiaokang Yang. Review on slam algorithms for augmented reality. *Displays*, 84:102806, 2024. 1
- [2] Nak-Jun Sung, Jun Ma, Kunthroza Hor, Taeheon Kim, Hongly Va, Yoo-Joo Choi, and Min Hong. Real-time physics simulation method for xr application. *Computers*, 14(1), 2025. 1
- [3] Richard A. Newcombe, Shahram Izadi, Otmar Hilliges, David Molyneux, David Kim, Andrew J. Davison, Pushmeet Kohi, Jamie Shotton, Steve Hodges, and Andrew Fitzgibbon. Kinectfusion: Real-time dense surface mapping and tracking. In *Proceedings of the 10th IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, pages 127–136. IEEE, 2011. 1
- [4] Taeheon Kim, Jun Ma, and Min Hong. Real-time cloth simulation in extended reality: Comparative study between unity cloth model and position-based dynamics model with gpu. *Applied Sciences*, 15(12), 2025. 1
- [5] Miles Macklin, Matthias Müller, Nuttapon Chentanez, and Tae-Yong Kim. Unified particle physics for real-time applications. *ACM Transactions on Graphics (TOG)*, 33(4):1–12, 2014. 1
- [6] Mohd Shahrizal Sunar, Abdullah Mohd Zin, and Tengku MT Sembok. Improved view frustum culling technique for real-time virtual heritage application. *Int. J. Virtual Real.*, 7(3):43–48, 2008. 1